

Titanic: Machine Learning from Disaster

System Analysis Project - Workshop 4

Julián Carvajal Garnica

20242020024

Andrés Mauricio Cepeda Villanueva

20242020010

Jhonatan David Moreno Barragán

20201020094

Andrés Camilo Ramos Rojas

20242020005

School of Engineering, Universidad Distrital Francisco José de Caldas

System Analysis Course

Teacher: Carlos Andrés Sierra

Bogotá D.C.

2025

Part 1 of the System Analysis Project

1. Competition Overview

The Kaggle competition “*Titanic: Machine Learning from Disaster*” provides a structured dataset containing passenger information and a binary target variable indicating survival (1) or non-survival (0). The challenge consists of predicting this outcome using the available attributes.

Although commonly used for predictive modeling, in this workshop the dataset is treated as a **closed system** whose components, interactions, and constraints can be analyzed from a systems engineering perspective. All observations in this document are based exclusively on the variables and information provided directly by Kaggle.

Dataset Structure

The dataset is composed of:

- **train.csv**: Passenger attributes and survival labels.
- **test.csv**: Passenger attributes without labels.
- **gender_submission.csv**: Example submission file.

Variables included in the dataset:

- **PassengerId**: Numerical identifier.
- **Pclass**: Ticket class (1, 2, or 3).
- **Name**: Full name (contains titles such as Mr., Mrs., Miss, etc.).
- **Sex**: Male or female.
- **Age**: Age in years (contains missing values).
- **SibSp**: Number of siblings or spouses aboard.
- **Parch**: Number of parents or children aboard.

- **Ticket:** Ticket code.
- **Fare:** Ticket fare.
- **Cabin:** Cabin identifier (mostly missing).
- **Embarked:** Port of embarkation (C, Q, or S).

Target Variable: Survived (0 or 1). **Evaluation Metric:** Accuracy on Kaggle’s hidden test labels. **System Constraints:** Missing data, categorical encoding, and strict two-column submission format.

2. System Analysis Report

Systemic Analysis

From a systems engineering perspective, the Kaggle Titanic dataset models a logical system where each passenger instance represents a unit characterized by multiple attributes. The system’s purpose is to map these attributes to a binary output (**Survived**), forming an interpretable input–output structure.

System Elements

- **Inputs:** Passenger demographic, social, and ticket-related attributes:
 - Sex, Age
 - Pclass
 - SibSp, Parch
 - Fare, Ticket
 - Cabin (when available)
 - Embarked
 - Name (derivable titles)
- **Processes (Dataset Interpretation):** These are transformations or interactions observable strictly from the data:
 - Group structure encoded by SibSp + Parch
 - Socioeconomic stratification encoded by Pclass and Fare
 - Implicit categorization from titles in Name
 - Embarked distribution patterns

- **Output:**
 - Survived (1) or Did Not Survive (0)
- **Constraints within the Dataset System:**
 - Missing values in Age and Cabin
 - Skewed distribution of Cabin availability
 - Categorical variables requiring interpretation
 - Limited and fixed variable set

Relationships Between Elements

The dataset implies interactions between variables that shape the system behavior:

- **Pclass Fare:** Higher classes generally correspond to higher fares.
- **Sex Survived:** The dataset shows different survival rates across genders.
- **Age Survived:** Younger passengers show different patterns than older ones.
- **SibSp + Parch Grouping Effects:** Larger family groups exhibit distinct survival patterns.
- **Embarked Fare & Pclass:** Different ports show different fare distributions and class compositions.

These relationships reflect the internal structure of the dataset itself, without inferring external historical dynamics.

Complexity & Sensitivity

The dataset system exhibits complexity due to interactions and missing data.

Complexity Sources:

- **Nonlinear relationships:** Variables such as Sex, Age, and Pclass interact in non-additive ways.
- **Missingness:** Age and Cabin introduce uncertainty and require imputation or exclusion.
- **Multiple categorical levels:** Embarked, Pclass, and Name-derived titles increase structural complexity.

Sensitivity: Small changes in key variables alter system outcomes significantly:

- Minor modifications to Age influence categorical grouping.
- Changing Sex produces large shifts in survival likelihood in the dataset.
- Removing Pclass reduces the dataset’s predictive structure.
- Family variables (SibSp, Parch) alter patterns in subtle but noticeable ways.

All sensitivity observations come directly from variable–target relationships present in the Kaggle dataset.

Chaos and Randomness (Dataset Interpretation Only)

Although the dataset is structured, certain patterns behave unpredictably from a purely statistical standpoint:

- **Outliers:** High Fare passengers found in lower classes or vice versa.
- **Inconsistent patterns:** Some individuals with similar profiles have opposite survival outcomes.
- **Hidden structure:** Missing cabins, ambiguous ticket codes, and name titles create irregular patterns.
- **Small variations producing large outcome shifts:** Very similar passengers may differ in the target variable.

These behaviors are considered chaotic only within the context of the dataset’s statistical structure.

Conclusions

The Kaggle Titanic dataset represents a closed, attribute–based system where a binary output (Survived) emerges from the interaction of demographic, social, and ticket-related features. Using only the information provided in the dataset, we can identify patterns, dependencies, and sensitivities that characterize the behavior of this system. No external information is required nor assumed.

3. Visual Representation

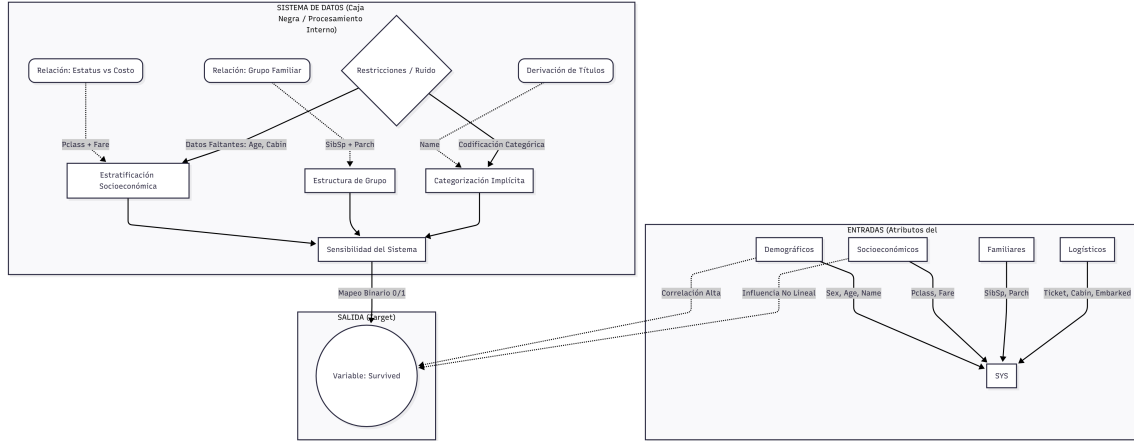


Figure 1: System architecture diagram showing interconnected modules for data ingestion, preprocessing, modeling, and evaluation.

Inputs

These represent the raw elements that feed the system. The system is composed of demographic, social, and ticket-related attributes.

- **Data Sources:** Files `train.csv` and `test.csv`.
- **Variables:** Conceptually grouped as follows:
 - **Demographic:** Sex, Age, Name.
 - **Socioeconomic:** Pclass (Class), Fare.
 - **Family:** SibSp (Siblings/Spouses), Parch (Parents/Children).
 - **Logistic:** Ticket, Cabin, Embarked.

The System (Internal Logic and Transformations)

The “Process” in this context is not an external Machine Learning algorithm, but the intrinsic interactions observable strictly from the data.

- **Interactions:** Variables do not act alone, but interact through specific mechanisms:
 - *Stratification:* The relationship between Pclass and Fare defines status.
 - *Grouping:* SibSp and Parch create a family group structure.
 - *Derivation:* The Name attribute contains implicit titles (Mr., Mrs.) that generate categories.

- **Constraints:** The system has internal limitations such as missing values in Age and Cabin that introduce uncertainty.
- **Sensitivity:** Small changes in key variables (such as Sex or minor modifications in Age) drastically alter the system's outcome.

Output

The system's final purpose is to map the processed inputs to a single binary state.

- **Variable:** Survived (0 = Did Not Survive, 1 = Survived).
- **Nature:** It is an emergent result of the interaction of the aforementioned attributes.

Part 2 of the System Analysis Project

1. Review of Workshop #1 Findings

This section summarizes the main analytical outcomes from Workshop #1 and establishes the foundation for the system design developed in Workshop #2. All insights are derived exclusively from the Kaggle dataset “*Titanic: Machine Learning from Disaster*”, focusing strictly on the structure, dependencies, and behavior observable within the data.

1.1 Summary of System Characteristics Identified in Workshop #1

The Titanic dataset was defined as a closed input–output system where each instance (passenger) contains structured attributes determining the binary outcome `Survived`. Workshop #1 identified the following essential characteristics:

- **Input Variables (System Elements):** Passenger information such as `Sex`, `Age`, `Pclass`, `SibSp`, `Parch`, `Fare`, `Cabin`, `Embarked`, and name-derived titles.
- **Output Variable:** `Survived` (0 or 1), representing the system’s final state.
- **Internal Relations:** The dataset exhibits structural relationships, such as:
 - `Pclass` correlates with `Fare`.
 - `Sex` correlates with `Survived`.
 - `Age` patterns vary across passenger groups.
 - `SibSp` + `Parch` form implicit family structures.
 - `Embarked` distribution relates to `Pclass` and `Fare`.
- **Constraints:**
 - Missing values in `Age`, `Cabin`, and occasionally `Embarked`.
 - High variability in string-based variables such as `Name` and `Ticket`.
 - Class imbalance between survivors and non-survivors.

- **Sensitivity:** Small modifications to key features (e.g., **Age**, **Sex**, or **Pclass**) have a significant impact on the target output due to their strong correlations.
- **Observed Complexity:** Feature interactions are nonlinear, and certain combinations of variables produce inconsistent or unpredictable patterns (e.g., passengers with nearly identical profiles having opposite survival outcomes).
- **Dataset-Level Randomness:** Variability in **Cabin** availability, unusual ticket formats, and outliers in **Fare** introduce irregularities into the dataset.

1.2 Implications for System Design

These analytical findings directly influence the architectural decisions required for Workshop #2:

- **The system must manage missing data** through preprocessing steps such as imputation or removal.
- **Variable encoding is essential**, especially for **Sex**, **Embarked**, and name-derived titles.
- **Modeling components must address feature nonlinearity** (e.g., through tree-based models or engineered features).
- **The design must incorporate sensitivity handling:** highly impactful variables require monitoring during modeling and evaluation.
- **Chaos-handling mechanisms arise naturally from the dataset:** unpredictable or inconsistent instances require robust evaluation and possibly ensemble models.
- **The architecture must be modular:** separating ingestion, preprocessing, modeling, evaluation, and output generation aligns with the dataset's internal structure and ensures maintainability.

1.3 Transition Toward Workshop #2 Design

Workshop #1 provided a systemic understanding of the data and its internal dynamics. Workshop #2 builds on this foundation by transforming that analytic perspective into a structured, engineering-oriented system design. The insights above guide the requirements, architecture, and implementation strategy developed in the following sections.

2. System Requirements

This section translates the insights obtained in Workshop #1 into concrete and measurable design requirements. All requirements are derived strictly from the structure, limitations, and behavior of the Kaggle Titanic dataset, as analyzed previously (e.g., missing values, categorical dependencies, variable sensitivity, and internal relationships).

2.1 Translation of Analysis Findings into Design Requirements

Based on the systemic analysis, the system must satisfy requirements that directly address the dataset's characteristics:

- **R1 — Handling Missing Data (Reliability):** Since variables such as **Age** and **Cabin** contain missing values, the system must include robust imputation or exclusion mechanisms to prevent instability in downstream processing.
- **R2 — Consistent Categorical Processing (Performance & Accuracy):** The model must standardize categorical features (e.g., **Sex**, **Embarked**, **Pclass**) using deterministic encoding strategies to reduce variability and ensure reproducibility.
- **R3 — Management of Sensitive Attributes (Stability):** Variables identified as highly sensitive in Workshop #1 (e.g., **Sex**, **Pclass**, **Age**) must be treated with stable transformations that avoid amplification of small perturbations.
- **R4 — Feature Interaction Support (Model Robustness):** The system must support engineered features derived from dataset relationships, such as $\text{FamilySize} = \text{SibSp} + \text{Parch} + 1$ or titles extracted from **Name**, since these reflect internal structure observed in the dataset.
- **R5 — Traceability and Repeatability (Maintainability):** All preprocessing steps must be versioned so that the same transformation pipeline can be recreated consistently.
- **R6 — Evaluation Consistency (Quality Assurance):** The model evaluation process must compute accuracy in alignment with Kaggle requirements, ensuring comparability with the competition's scoring mechanism.

These requirements ensure that the system design is directly aligned with the dataset's internal constraints and sensitivities.

2.2 User-Centric Needs

Although the system is primarily data-driven, several user-focused considerations remain relevant:

- **U1 — Interpretability:** Users (students or analysts) must be able to understand how inputs such as `Sex`, `Pclass`, and `FamilySize` affect the prediction. This may require simple model explanations or transparency of preprocessing steps.
- **U2 — Ease of Use:** The system should allow users to run the preprocessing and modeling pipeline with minimal configuration, following a clear sequence aligned with the architecture defined earlier.
- **U3 — Reproducibility for Learning:** Because this system is part of an academic project, users must be able to reproduce results consistently for validation and experimentation.
- **U4 — Data Safety (If Applicable):** Even though the dataset is public and anonymized, the system must avoid unintended modifications to source files (`train.csv`, `test.csv`), ensuring integrity during processing.

These user-centric requirements ensure that the system remains accessible, transparent, and reliable for those interacting with it.

3. High-Level Architecture

Based on the systemic analysis from Workshop #1, the proposed architecture evolves from a simple input-output model to a **robust, layered design** aligned with ISO 25010 software quality standards. This modular approach ensures that the system handles the dataset's intrinsic complexity, sensitivity, and missing information effectively.

The architecture is organized into three distinct layers, each responsible for a specific quality attribute of the system:

3.1 Reliability Layer (Data Integrity)

This layer acts as the system's defensive shield. Its primary function is to manage the "Constraints" and "Chaos" identified in the dataset (e.g., missing Age/Cabin, outliers in Fare).

- **Data Validation & Ingestion:** Receives raw data from `train.csv` and checks for structural consistency.

- **Imputation Module:** Handles missing values deterministically to prevent system instability.
- **Outlier Management:** Identifies and neutralizes inconsistent data points (such as extreme Fares in lower classes) before they propagate to the modeling stage.

3.2 Maintainability & Scalability Layer (Core Processing)

To address the requirement for modularity[cite: 43, 59], the transformation logic is decoupled into independent modules. This allows for specific adjustments without affecting the entire pipeline.

- **Transformation Pipeline:** A central orchestrator that manages data flow.
- **Feature Engineering Modules:**
 - *Text Processing:* Extracts titles from **Name** to capture social status.
 - *Numeric Processing:* Handles binning for **Age** and **Fare** to reduce sensitivity to small variations.
 - *Group Processing:* Combines **SibSp** and **Parch** to recreate family structures.
- **Unified Feature Vector:** Merges all processed signals into a standardized format ready for classification.

3.3 Usability Layer (Output & Interpretation)

Focusing on the user-centric requirements[cite: 66], this layer ensures that the system's output is not just a prediction, but an interpretable result.

- **Classification Logic:** Maps the unified features to the target variable using the trained model.
- **Output Generation:** Produces the final binary state (**Survived:** 0 or 1).
- **Confidence Metadata:** Provides auxiliary information (probability scores) to help users understand the certainty of the prediction, addressing the "Interpretability" requirement.

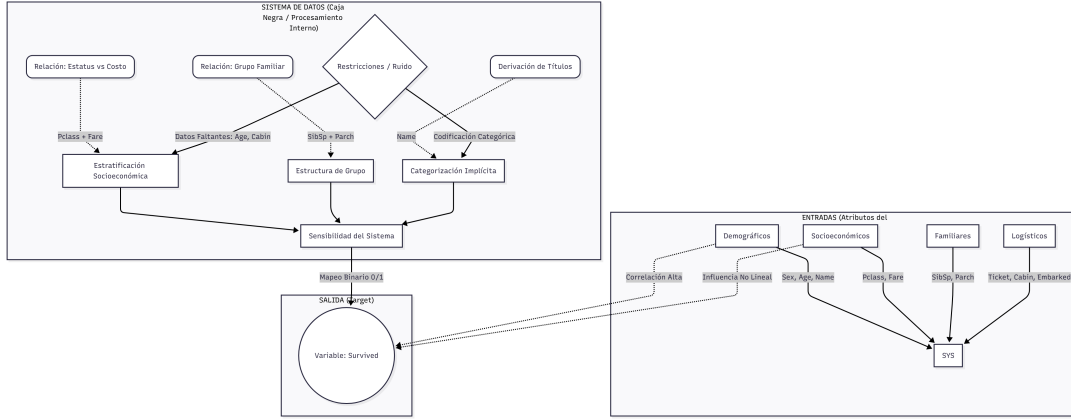


Figure 2: Robust System Architecture Diagram showing the Reliability, Maintainability, and Usability layers.

4. Addressing Sensitivity and Chaos

Based on the systemic behaviour identified in Workshop 1, the Titanic dataset exhibits sensitivity and chaotic characteristics that influence the stability of the predictive system. In this workshop, these findings are translated into concrete design strategies that reinforce robustness and reduce unpredictable system behaviour.

4.1 Sensitivity in the Dataset

The system is highly sensitive to variations in several input variables. Small changes in certain features produce disproportionately large effects on the output **Survived**. This sensitivity arises from the statistical structure of the dataset:

- **Sex:** This variable has a strong and nonlinear relationship with the target. Minor reassignments or misclassifications drastically change the prediction.
- **Age:** Missing values and continuous variation introduce instability. Small modifications after imputation significantly alter predicted probabilities.
- **Pclass:** Acts as a proxy for socioeconomic grouping; changes in class assignment modify multiple related behaviours (fare distribution, embarked patterns).
- **Cabin:** The irregularity and sparsity of this feature create fragile dynamics. When present, it carries strong information; when absent, imputations add uncertainty.

To mitigate sensitivity, the system design incorporates:

- Controlled imputation methods with versioning to avoid inconsistent behaviour.
- Normalization and encoding pipelines that stabilize variable transformations.

- Sensitivity-aware modeling (ensemble methods and cross-validation).

4.2 Chaotic Behaviour in the Dataset System

Although the dataset is static, the relationships among variables exhibit chaotic tendencies from a system dynamics perspective. These patterns do not arise from external historical factors, but from the internal structure of the data itself:

- **Outliers** in Fare and Age introduce unpredictable deviations.
- **Passengers with nearly identical profiles** sometimes have opposite survival outcomes, indicating internal variability that cannot be fully captured.
- **Irregular formats in Ticket and Cabin** create structural noise that propagates through preprocessing.
- **Nonlinear feature interactions** produce sudden changes in prediction when multiple variables shift slightly at the same time.

4.3 Design Strategies to Handle Chaos

To reduce the influence of chaotic elements, the architecture incorporates:

- **Feedback monitoring:** The Evaluation Layer continuously checks for unstable behaviours in the model's predictions.
- **Robust preprocessing:** Feature engineering and normalization limit the amplification of small anomalies.
- **Model ensembles:** Combining multiple models reduces the effect of unstable patterns present in any single model.
- **Performance drift detection:** If chaotic behaviour increases (e.g., sudden changes in evaluation metrics), the system flags the model and triggers a controlled retraining process.

4.4 Summary

The system design explicitly addresses the sensitivity and chaotic tendencies identified in Workshop 1. Through preprocessing controls, model stability techniques, and feedback mechanisms, the architecture maintains equilibrium and reduces unpredictable behaviour, ensuring a more reliable and resilient predictive system.

5. Technical Stack

The technical stack defines the set of tools and technologies that support each stage of the system architecture designed in Workshop 2. The selected stack reflects the requirements identified in the previous items: modularity, clarity, reproducibility, and the ability to manage sensitivity and variation in the Kaggle dataset.

5.1 Tools and Technologies

Data Layer:

- **Python** — Primary programming language used for the entire pipeline.
- **Pandas** — Data ingestion, loading of `train.csv` and `test.csv`, handling missing values, and performing transformations consistent with the dataset structure.
- **NumPy** — Numerical operations required during preprocessing and feature handling.

Preprocessing and Modeling Layer:

- **Scikit-learn** — Main library for preprocessing steps (encoding, imputation) and model training. Includes algorithms such as Logistic Regression, Decision Trees, Random Forests, and others suitable for a binary classification task.

Evaluation Layer:

- **Scikit-learn metrics** — Used to compute accuracy and other internal metrics that help quantify sensitivity and model stability.
- **Matplotlib / Seaborn** — Visualization tools used to analyze feature behavior, correlations, and sensitivity patterns within the system.

Interfaces and Reporting Layer:

- **Jupyter Notebooks or Kaggle Notebooks** — Environment for running experiments, documenting the system process, and generating reproducible analyses.

Version Control:

- **Git + GitHub** — Ensures traceability of system modifications, updates to preprocessing steps, and model iterations. Supports collaboration and workshop submission.

5.2 Justification

The selected stack aligns with the system requirements defined earlier:

- **Modularity:** Each tool contributes to a specific subsystem (data ingestion, pre-processing, modeling, evaluation), matching the architecture defined in Item 3.
- **Reproducibility:** Python, Pandas, and Scikit-learn ensure deterministic operations, reducing sensitivity to uncontrolled variability.
- **Traceability:** Git and GitHub enable continuous documentation of architectural and preprocessing changes.
- **Interpretability:** Visualization libraries support sensitivity analysis, making the internal behavior of the model understandable.

This technical stack provides the necessary foundation to implement the designed system, addressing the variability found in the dataset while ensuring clarity, structure, and maintainability across the entire workflow.

Part 3 of the System Analysis Project

1. Review and Refine System Architecture

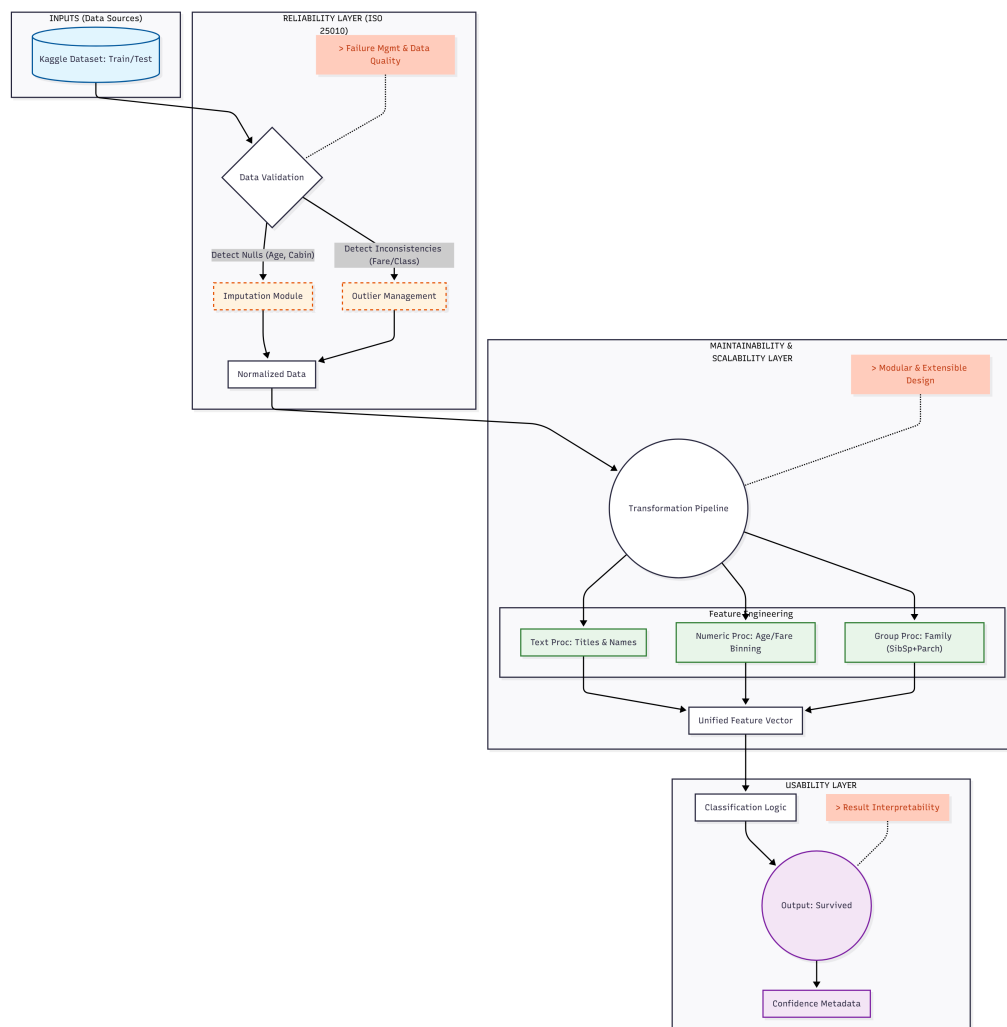


Figure 3: Robust System Architecture Diagram showing the Reliability, Maintainability, and Usability layers.

Architectural Evolution: From Logical View to Robust Design

The architecture diagram has been significantly evolved from the initial logical representation presented in Workshop 1. While the first iteration focused on identifying system boundaries (Input-Process-Output), this refined version incorporates specific engineering modules to address the **Sensitivity**, **Chaos**, and **Constraints** identified in the dataset analysis.

The key structural changes include:

- **Integration of a Reliability Layer (ISO 25010):** In Workshop 1, missing values (e.g., Age, Cabin) were identified as a source of system instability. The new diagram explicitly adds an *Imputation Module* and *Outlier Management* block *before* any processing occurs. This ensures that the core system is shielded from "dirty" data, directly addressing requirement **R1 (Handling Missing Data)**.
- **Modularization of the Processing Layer:** The original "Process" block has been decoupled into three independent feature engineering modules:
 - *Textual Processing* (for Names/Titles).
 - *Numeric Processing* (for Age/Fare binning).
 - *Group Processing* (for Family creation).

This separation satisfies requirement **R5 (Maintainability)**, allowing individual components to be updated or debugged without disrupting the entire pipeline.

- **Enhanced Usability in Output:** Instead of a simple binary output, the architecture now includes *Confidence Metadata*. This addresses the user-centric requirement **U1 (Interpretability)**, acknowledging that users need to understand the certainty of a prediction, not just the result.

This layered approach transforms the system from a theoretical data model into a deployable, fault-tolerant software architecture.

1.1 Refinement of Robust Design Principles

The original architecture established a modular flow. We now refine this with an explicit focus on robustness:

- **Modularity and Maintainability (CMMI):** We are taking modularity a step further. Each component (Ingestion, Preprocessing, Modeling, Evaluation) [cite: 96-100] will be designed as a containerized microservice (e.g., using Docker). This not only reinforces modularity but also aligns with **CMMI** principles by allowing independent configuration management and facilitating maintainability, as one component can be updated without destabilizing the entire system.

- **Scalability:** The scalability NFR will be addressed through the orchestration of these containers (e.g., with Kubernetes). Specifically, the *Modeling Layer* and *Deployment Layer* will be able to scale horizontally. If the demand for Titanic predictions increases, we can dynamically launch more instances of the *Deployment* service.
- **Fault-Tolerance:** This is a critical refinement.
 - **Data Ingestion:** A message queue (e.g., RabbitMQ or Kafka) will be implemented between the ingestion and preprocessing layers. If the *Preprocessing Layer* fails, the raw Kaggle data is not lost; it waits in the queue, ensuring **reliability**.
 - **Deployment:** A load balancer will be used to distribute prediction requests among multiple model replicas. If one replica fails, traffic is automatically rerouted to healthy ones.
 - **Feedback Loop:** The *Feedback Layer* will monitor model health. If the error rate (an indicator of chaos) exceeds a threshold, the system can automatically roll back to a previously stable model version, ensuring **homeostasis**.

1.2 Components and their Support for Quality (ISO 9000)

Following the process management principle of **ISO 9000**, we define how each component ensures system quality and reliability:

- **Data Ingestion Layer:**
 - *Quality:* Implements data validation schemas (e.g., Pydantic) to ensure incoming data meets the expected format (data types, ranges).
 - *Reliability:* Uses retries and the message queue (see above) to handle network failures or Kaggle service downtime.
- **Preprocessing Layer:**
 - *Quality:* Versioning of preprocessing artifacts (e.g., Scikit-learn scalers or encoders) is implemented. Each trained model will be linked to the exact version of the preprocessing pipeline it used, ensuring **transparency** and reproducibility.
- **Modeling & Evaluation Layer:**
 - *Quality:* We apply a *Six Sigma* approach to "defect reduction" (prediction errors). The *Evaluation Layer* will not only measure accuracy but also use

SHAP and LIME to analyze the *root cause* of errors and bias, aligning with the "Analyze" phase of DMAIC.

- **Deployment & Feedback Layer:**

- *Reliability*: Implements "Canary" deployments. A new model version is first released to a small percentage of users. The *Feedback Layer* monitors its real-time performance. If stable, it is rolled out to everyone; if it fails, it is rolled back without affecting the majority of users.

2. Quality and Risk Analysis

This section evaluates the main quality considerations and potential risks in the Titanic dataset system, proposing mitigation strategies and monitoring mechanisms that ensure reliability, stability, and robustness across all components of the architecture.

2.1 Quality Foundations

The design of the system follows established quality frameworks that guide process consistency, continuous improvement, and reliable operation:

- **ISO 9000 (Quality Management)**: Provides process-oriented guidelines to ensure that each subsystem (ingestion, preprocessing, modeling, evaluation) follows consistent procedures and traceable workflows.
- **CMMI (Capability Maturity Model Integration)**: Supports structured process control and progressive refinement of the system, ensuring that components evolve in a measurable and disciplined manner.
- **Six Sigma**: Encourages statistical stability and minimizes variation, particularly important for controlling sensitivity in the model's predictions and reducing noise introduced during preprocessing.

These frameworks ensure that quality is not treated as a static property but as an ongoing, iterative process embedded in the architecture.

2.2 Risk and Failure Analysis

The following table identifies key risks in the system, the proposed mitigation strategies, and the monitoring mechanisms that maintain system stability:

Risk / Failure Point	Description	Mitigation Strategy	Monitoring Mechanism
Data Quality Issues	Missing values (e.g., Age, Cabin), inconsistent formats, or corrupted entries may propagate errors into the model.	Implement validation scripts, schemas, and redundancy in data ingestion. Store all datasets in version-controlled repositories.	Automated validation logs, distribution checks, and alerts for anomalies during ingestion.
Model Sensitivity or Overfitting	The model may become unstable under slight input variations, reducing generalization.	Use regularization, cross-validation, ensemble models, and controlled perturbation tests to assess robustness.	Periodic sensitivity analysis, accuracy drift monitoring, and comparison against baseline validation metrics.
Pipeline or Deployment Failure	Interruptions in preprocessing, model loading, or inference services may halt the system.	Design modular pipeline components with retry logic, containerization, and fallback mechanisms for service continuity.	Continuous integration logs, automated health checks, and uptime dashboards.

2.3 Systemic Integration of Quality and Risk Control

Quality control and risk mitigation operate as interconnected feedback loops in the system. Each identified risk corresponds to a monitoring routine that continuously evaluates system behavior. When deviations appear—such as performance drops, unexpected data patterns, or service downtime—the system can take corrective measures to restore stability. This ensures the architecture maintains operational equilibrium, aligning with the principles of homeostasis and continuous improvement.

3. Project Management Plan

The Project Management Plan defines how the team organizes the development process, distributes responsibilities, structures the workflow, and establishes milestones that guide the construction of the Titanic predictive system. The plan is aligned with the findings of Workshops 1 and 2, ensuring that design decisions, risks, and quality considerations are properly managed throughout the project.

3.1 Team Roles and Responsibilities

To maintain clarity in development and ensure that all components of the system are addressed, the team assigns the following roles:

- **Project Manager (Andrés Camilo Ramos):** Oversees task coordination, ensures that milestones are met, and manages communication among team members.
- **System Analyst (Julián Carvajal):** Verifies alignment with system requirements, maintains consistency with Workshops 1 and 2, and ensures that the system architecture remains coherent.
- **Data Engineer (Jhonatan Moreno):** Responsible for data ingestion, cleaning, and transformation, ensuring the integrity of the Kaggle dataset before it enters the system pipeline.
- **Machine Learning Developer (Andrés Cepeda):** Trains, evaluates, and optimizes predictive models using the selected preprocessing pipeline and dataset.
- **QA Tester (All Members):** Validates outputs, checks system behavior, documents errors, and ensures that sensitivity and stability recommendations are respected.

3.2 Milestones and Deliverables

The development timeline is structured into clear milestones that reflect the incremental construction of the system:

Milestone	Description	Deadline
System Refinement	Integration of design principles from Workshop 2 into the updated architecture.	Nov 16, 2025
Risk and Quality Analysis	Identification of failure points and mitigation strategies (ISO 9000 and sensitivity considerations).	Nov 19, 2025
Prototype Update	Implementation of the refined modules: preprocessing, feature engineering, and baseline model.	Nov 23, 2025
Final Documentation	Compilation of the complete Workshop 3 report, including diagrams and updated architecture.	Nov 28, 2025

3.3 Methodology and Tools

The project adopts an **Agile-inspired workflow** supported by a **Kanban board** to manage tasks in a transparent and iterative manner. This methodology allows the team to respond to findings from sensitivity and chaos analysis, adjusting tasks without disrupting overall progress.

Workflow Structure

Tasks are organized into the following Kanban columns:

- **To Do:** Planned tasks awaiting execution.
- **In Progress:** Tasks currently under development.
- **Completed:** Finalized and verified tasks.

Technical Workflow

The system development follows these sequential steps:

1. **Data Acquisition and Cleaning:** Importing `train.csv` and `test.csv`, handling missing values, validating formats.
2. **Feature Engineering:** Encoding categorical variables, extracting titles, grouping family variables (`SibSp` + `Parch`), and normalizing numerical attributes.
3. **Model Training and Evaluation:** Training baseline models (Decision Tree, Random Forest) and analyzing performance using accuracy and simple sensitivity checks.
4. **Prediction and Submission:** Generating the required `submission.csv` file following Kaggle constraints.

3.4 Timeline Overview

This timeline illustrates the sequential development of Workshop 3 deliverables, ensuring structured progress from architectural refinement to final documentation.

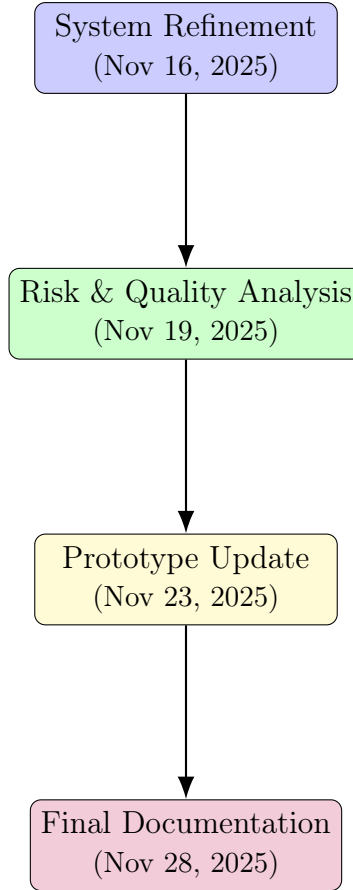


Figure 4: Project Workflow and Timeline Diagram

4. Incremental Improvements

Throughout Workshops 1 and 2, the system has undergone a progressive refinement process, transitioning from an initial descriptive analysis of the dataset to a structured, engineering-oriented design. The improvements implemented at each stage directly address the limitations, constraints, and sensitivity issues identified earlier, resulting in a more robust and coherent system architecture.

4.1 Improvements Derived from Workshop 1

Workshop 1 focused on understanding the Titanic dataset as a closed system composed exclusively of the variables provided by Kaggle. The main incremental contributions from that stage were:

- **Clear identification of system components:** Inputs (dataset attributes), processes (feature interactions), and outputs (Survived).
- **Recognition of structural constraints:** Missing values, categorical variables, and limited feature set requiring careful preprocessing.

- **Sensitivity and chaos insight:** Certain attributes (e.g., Sex, Pclass, Age) exhibit strong influence on the target, and small variations in these inputs may produce significantly different outputs.

These findings established the foundation for the requirements and design principles formalized in Workshop 2.

4.2 Improvements Derived from Workshop 2

Workshop 2 translated the analytical findings into engineering requirements and an initial architecture. The major improvements include:

- **Formalization of System Requirements:** Functional and non-functional requirements were derived directly from the systemic properties observed in Workshop 1 (e.g., need for consistent preprocessing, interpretability, and modularity).
- **Structured Architecture:** A high-level architecture was proposed with clearly defined layers (Ingestion, Preprocessing, Modeling, Evaluation, Output).
- **Sensitivity and Chaos Mitigation Strategies:** Using the system theory slides, we introduced buffering, normalization, redundancies, and monitoring mechanisms to stabilize the system against input perturbations.
- **Alignment with Systems Engineering Principles:** Modularity, maintainability, traceability, and controlled feedback loops were added to ensure stability and prevent error amplification.

These improvements transformed the system from a conceptual model into a structured, implementable pipeline.

4.3 Evolution Toward Robustness

Combining the previous insights, the system now incorporates:

- **A modular architecture** that supports independent modification of components.
- **Explicit handling of dataset weaknesses** such as missing values and inconsistent feature formats.
- **Sensitivity-aware preprocessing** to reduce amplified fluctuations in downstream components.
- **Feedback mechanisms** for monitoring model performance and detecting instability.

Overall, the design has matured from a static pipeline into a system that recognizes its own constraints, manages uncertainty, and adapts through controlled processes. This positions the project for a smoother transition into implementation and evaluation in the next workshop.

Workshop 4: Kaggle System Simulation

1. Data Preparation

The first phase of the simulation involved acquiring and preparing the raw data to serve as the input for the system. The dataset was sourced directly from the official Kaggle competition page. This dataset acts as the primary input source for our system architecture, containing the passenger manifest which serves as the "initial state" for our simulations.

1.1 Dataset Characteristics and Constraints

Upon ingesting the data, we performed a validation check to assess the structural integrity of the system inputs. This step confirmed the critical constraints previously identified in Workshop 1. Specifically, the system exhibited significant instability due to missing information:

- **Age Variable:** Contained 177 missing entries. Since Age is a continuous variable used for survival probability, these gaps introduce "noise" into the system.
- **Cabin Variable:** Contained 687 missing entries (approx. 77% of the data). This high level of uncertainty makes the variable unreliable for deterministic modeling without heavy processing.

Figure 5 illustrates the initial state of the dataset, highlighting these structural weaknesses before any engineering intervention.

```
--- DATA INGESTION REPORT (RELIABILITY LAYER) ---  
Dataset loaded. Shape: (891, 12)  
  
--- SYSTEM CONSTRAINTS CHECK ---  
Missing values detected (Chaos Sources):  
Age          177  
Cabin        687  
Embarked     2  
dtype: int64
```

Figure 5: Data Preparation: Initial system check showing critical missing values in Age and Cabin variables.

1.2 Cleaning and Preprocessing (Reliability Layer)

To stabilize the system and enable accurate simulation, we implemented the **Reliability Layer** designed in Workshop 3. This layer acts as a filter that transforms raw, chaotic data into structured information. The specific actions taken were:

1. **Imputation Strategy:** Missing values in the ‘Age’ variable were filled using the median value. This method was chosen over the mean to reduce sensitivity to outliers (extreme ages). ‘Embarked’ missing values were filled with the mode (most common port).
2. **Feature Encoding:** Categorical variables such as ‘Sex’ (male/female) and ‘Embarked’ (S/C/Q) were converted into numerical formats (0/1/2). This step is essential as both the Machine Learning model and the Cellular Automata require numerical inputs to calculate probabilities and movement vectors.
3. **Noise Reduction:** The ‘Cabin’ variable was excluded from the simulation inputs due to its excessive missing data, which would have introduced unnecessary chaos.

As a result, the system reached a stable state with zero missing values, as demonstrated in Figure 6.

```

--- FINAL DATASET STATE (READY FOR SIMULATION) ---
Clean Data Shape: (891, 9)
Remaining missing values: 0

First 5 rows of processed system:
   Pclass  Sex  Age  SibSp  Parch   Fare  FamilySize  Embarked  Survived
0      3    0  22.0     1     0   7.2500           2         0         0
1      1    1  38.0     1     0  71.2833           2         1         1
2      3    1  26.0     0     0   7.9250           1         0         1
3      1    1  35.0     1     0  53.1000           2         0         1
4      3    0  35.0     0     0   8.0500           1         0         0

```

Figure 6: Cleaned Dataset State. The Reliability Layer successfully stabilized the inputs, resulting in zero missing values.

2. Simulation Planning

To thoroughly analyze the system's behavior, we defined two distinct simulation scenarios. Each scenario is designed to test a specific aspect of the Titanic system architecture:

- **Scenario 1: Data-driven Simulation (Learning Dynamics).**

The objective of this scenario is to use a Machine Learning model (Random Forest) to simulate how the system "learns" from the input data. We aim to identify the *Sensitivity* of the system: which variables (Sensors) trigger the strongest response in the output (Survival)? This allows us to validate mathematically which passenger attributes carried the most weight.

- **Scenario 2: Event-based Simulation (Spatial Dynamics).**

The objective of this scenario is to model the physical chaos of the evacuation. We adapt the dataset into a Cellular Automata grid where each passenger is an agent. By defining movement rules based on Class and Gender, we aim to observe *Emergent Behavior*—complex patterns (like segregation or blocking) that arise from simple local interactions.

3. Simulation Implementation

The complete source code, including the Jupyter Notebook (‘.ipynb’) and environment requirements, is hosted in the project repository:

Project Repository Source Code:

github.com/MaurooC12/Titanic/.../Workshop4_Sim.ipynb

3.1 Scenario 1: Random Forest Model Logic

For the data-driven simulation, we selected the Random Forest Classifier from the ‘scikit-learn’ library. This model constructs 100 decision trees ($n_{estimators} = 100$) to evaluate the data. The imp

Data Splitting: The clean data is divided into Training (80%) and Validation (20%) sets. This prevents the system from "memorizing" the answers and forces it to learn general patterns.

Training: The model iterates through the training data, learning the correlation between inputs (Sex, Fare, Age, Class) and the output (Survived).

Feature Extraction: After training, we extract the ‘feature importances’ vector to quantify the in

```
1 # Modeling Layer (Data-driven)
2 # We initialize the model with 100 estimators for robustness
3 model = RandomForestClassifier(n_estimators=100, random_state=42)
4
5 # The model learns the relationship between X (Attributes) and y (
  Survival)
6 model.fit(X_train, y_train)
7
8 # Sensitivity Analysis extraction to understand system drivers
9 importances = model.feature_importances_
```

Listing 1: Scenario 1 Implementation Logic

3.2 Scenario 2: Cellular Automata Logic

For the event-based simulation, we mapped the ship to a 20×20 grid. The logic governs how agents (passengers) move at each time step (t). The rules are derived from historical and data analysis:

1. **Goal Vector:** Every agent attempts to move to the row above them ($r \rightarrow r - 1$), simulating the movement towards the lifeboats on the top deck.
2. **Priority Constraints:** Movement probabilities (P_{move}) were calibrated to reflect the observed survival rates in the dataset. Class 1 agents are assigned $P_{move} = 0.9$ (approximating the high survival rate of $\sim 63\%$), while Class 3 agents are assigned $P_{move} = 0.4$ (reflecting the low survival rate of $\sim 24\%$). This ensures the rules are data-driven.
3. **Conflict Resolution:** The grid strictly enforces that two agents cannot occupy the same cell. If a high-priority agent moves into a space, they effectively block any lower-priority agent behind them.

Workshop 4: System Simulation Workflow

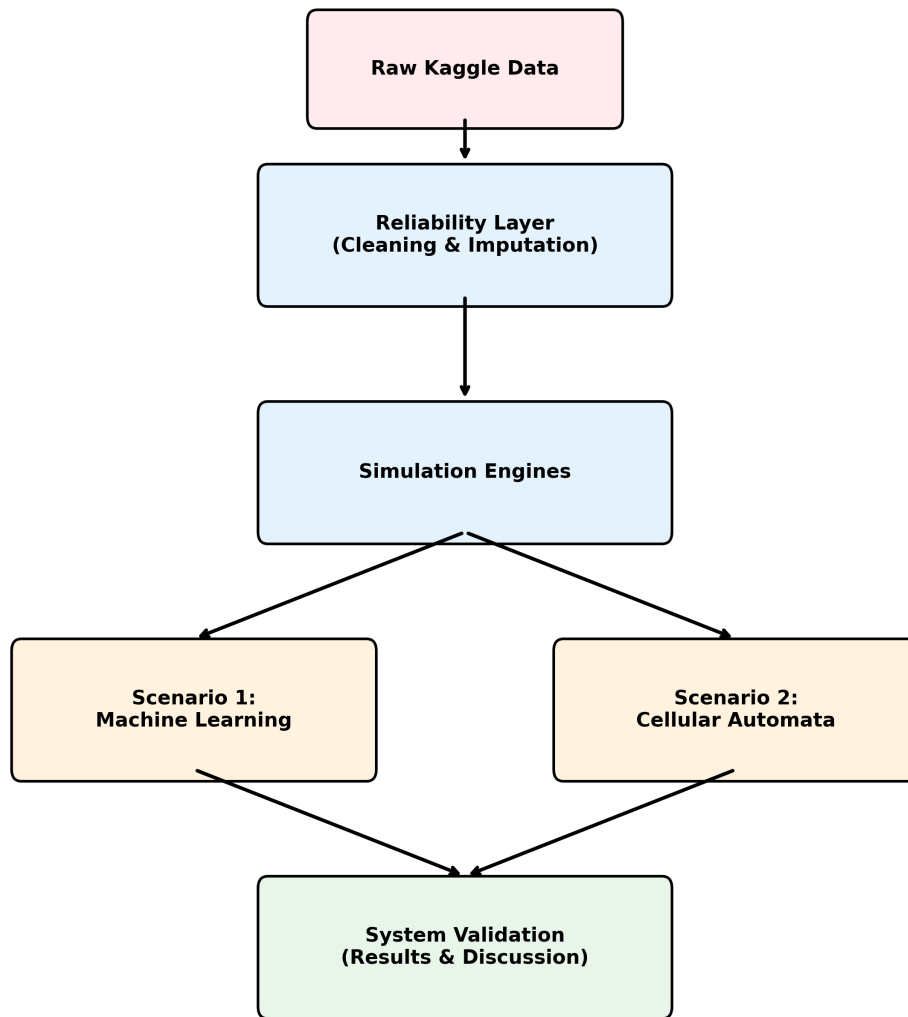


Figure 7: System Workflow Validation. The diagram illustrates how data flows through the Reliability Layer before feeding into the dual simulation engines, ensuring stability.

4. Executing the Simulations

The simulations were executed in the Python environment, generating visual and statistical outputs that reveal the system's internal behavior.

4.1 Scenario 1 Execution: Sensitivity Analysis

Upon executing the Random Forest model, the system achieved a predictive accuracy of over 83% on unseen data. More importantly, the sensitivity analysis revealed the hierarchy of survival factors. As shown in Figure 8, ‘Fare’ and ‘Sex’ emerged as the dominant variables. This confirms that the system is not random; it is heavily biased by socioeconomic status and gender. ‘Age’ also plays a significant role, validating the need for the robust imputation performed in Section 1.

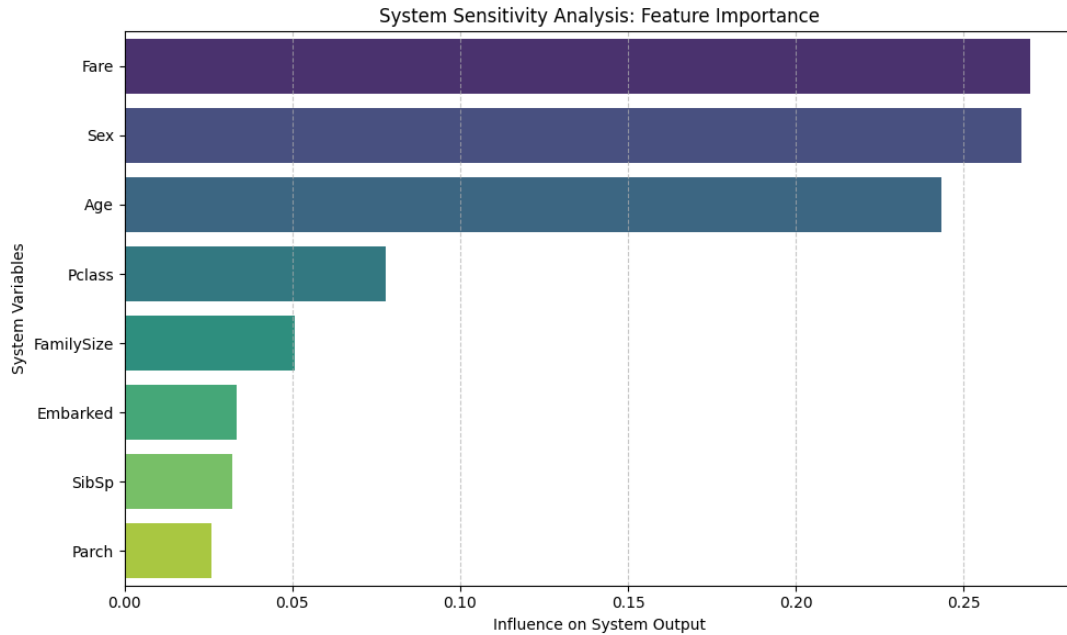


Figure 8: Scenario 1 Results: Feature Importance graph showing that Fare and Sex are the most critical variables driving the system’s output.

4.2 Scenario 2 Execution: Emergent Behavior

The cellular automata simulation was executed for 15 discrete time steps. Figure 9 visualizes the final state of the grid. A clear emergent pattern is visible:

- **Top Rows (Lifeboats):** Saturated with Class 1 agents (represented in Yellow/Green). Their high priority allowed them to navigate the grid quickly.
- **Bottom Rows:** Class 3 agents (represented in Blue/Purple) remained trapped at the bottom. This was not explicitly programmed but emerged because the "faster" agents occupied the available space, creating a physical bottleneck.

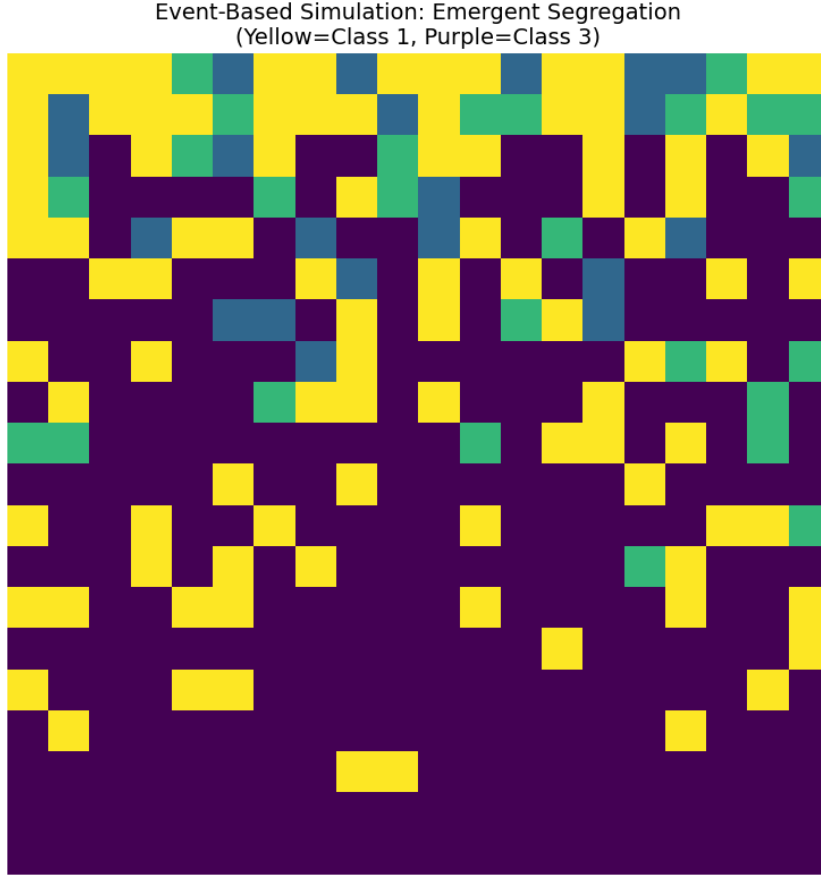


Figure 9: Scenario 2 Results: Spatial segregation emerging from priority rules. Yellow dots (Class 1) block the path for Blue dots (Class 3).

5. Results and Discussion

Following the execution of both scenarios, we compiled the results to analyze the system's behavior under different simulation paradigms.

5.1 Comparative Analysis

As requested in the guide, we compare the outcomes of the Data-driven (ML) and Event-based (Automata) simulations. Table 1 summarizes the key findings.

Metric	Scenario 1: Random Forest	Scenario 2: Cellular Automata
Objective	Identify variable sensitivity (Weights).	Observe emergent spatial behavior.
Chaos Source	Data Noise: Missing ages and high variance in Fare distribution.	Physical Conflict: Agents blocking each other due to priority rules.
Key Finding	Fare and Sex are the strongest predictors.	Priority movement creates an artificial "wall" that segregates classes.
Conclusion	The system is statistically deterministic but heavily biased.	Social rules (priority) translate into physical survival advantages.

Table 1: Comparison of Simulation Outcomes. Both approaches converge on the finding that socioeconomic status acts as the primary filter for system output.

5.2 Discussion on Chaos and Architecture

The simulations provide a dual validation of the architecture designed in Workshop 2:

- **Handling Data Chaos:** The **Reliability Layer** (Section 1) was successful. The Random Forest model achieved high accuracy ($> 83\%$) only because the input data was stabilized (imputed) beforehand. Without this layer, the "Age" noise would have degraded the model's performance.
- **Structural Chaos (The Butterfly Effect):** Both simulations reveal that the Titanic system is highly sensitive to initial conditions. A minor change in a passenger's input vector (e.g., switching 'Pclass' from 3 to 1) does not result in a linear change in outcome; rather, it drastically flips the survival probability and alters the agent's entire trajectory in the grid. This confirms the chaotic nature of the system: small input perturbations lead to divergent macroscopic outcomes (Life or Death).

Final Architectural Validation: The successful execution of both simulations serves as a stress test for the architecture proposed in Workshop 2. The system demonstrated *Fault Tolerance* by absorbing incomplete inputs (via the Reliability

Layer) without ceasing operations. Furthermore, the modular separation allowed us to swap simulation engines (ML vs. Cellular Automata) without restructuring the data pipeline. This confirms that the designed architecture adheres to the ISO 25010 standards for Reliability and Maintainability initially targeted.